

Onaylı Kanal Dışına Çıkış Yok

PTC (Programmatic Tool Calling) ile çalışan bir kurumsal asistanın, ürettiği kodun ağ erişimini eBPF/Cilium ile kernel seviyesinde nasıl kısıtladığını gösteren bir kavram kanıtı.

2

katmanlı Cilium egress politikası —
sandbox ve Tool Gateway ayrı ayrı kısıtlı

3

onaylı dış hedef — isim bazlı (SNI),
geri kalan her yer kapalı

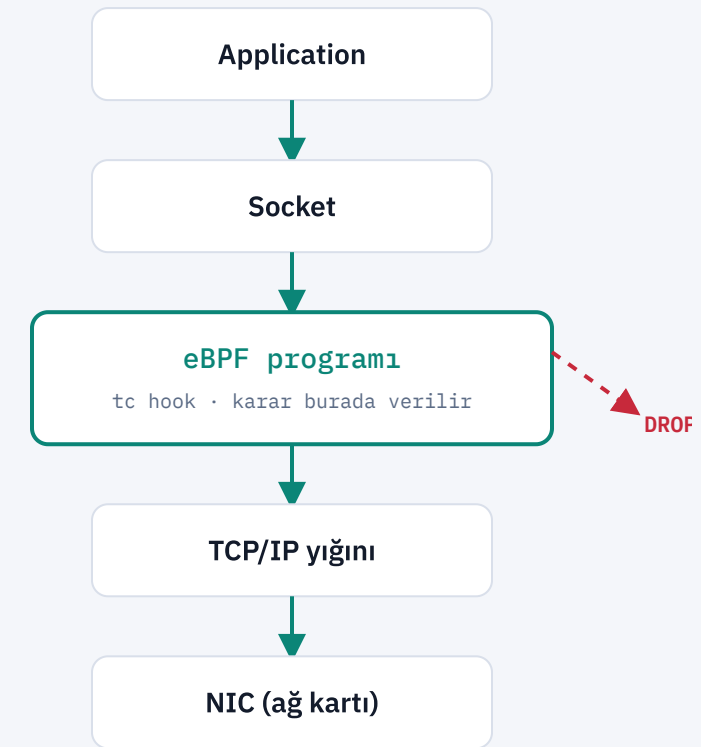
0

başarılı kaçış — canlı olarak,
kernel sayaçlarıyla doğrulandı

eBPF Nedir

Linux kernel'inin içine güvenli (sandboxed), küçük programlar takmayı sağlayan genel amaçlı bir motor — kernel modülü yazıp kernel'i yeniden derlemeden. Ağa ya da Kubernetes'e özgü değil.

eBPF, kernel'in birden çok noktasına (sistem çağırısı, ağ paketi, fonksiyon izleme) program takılmasına izin verir — bizim hikâyemizdeki nokta: ağ paketi (tc hook'u).

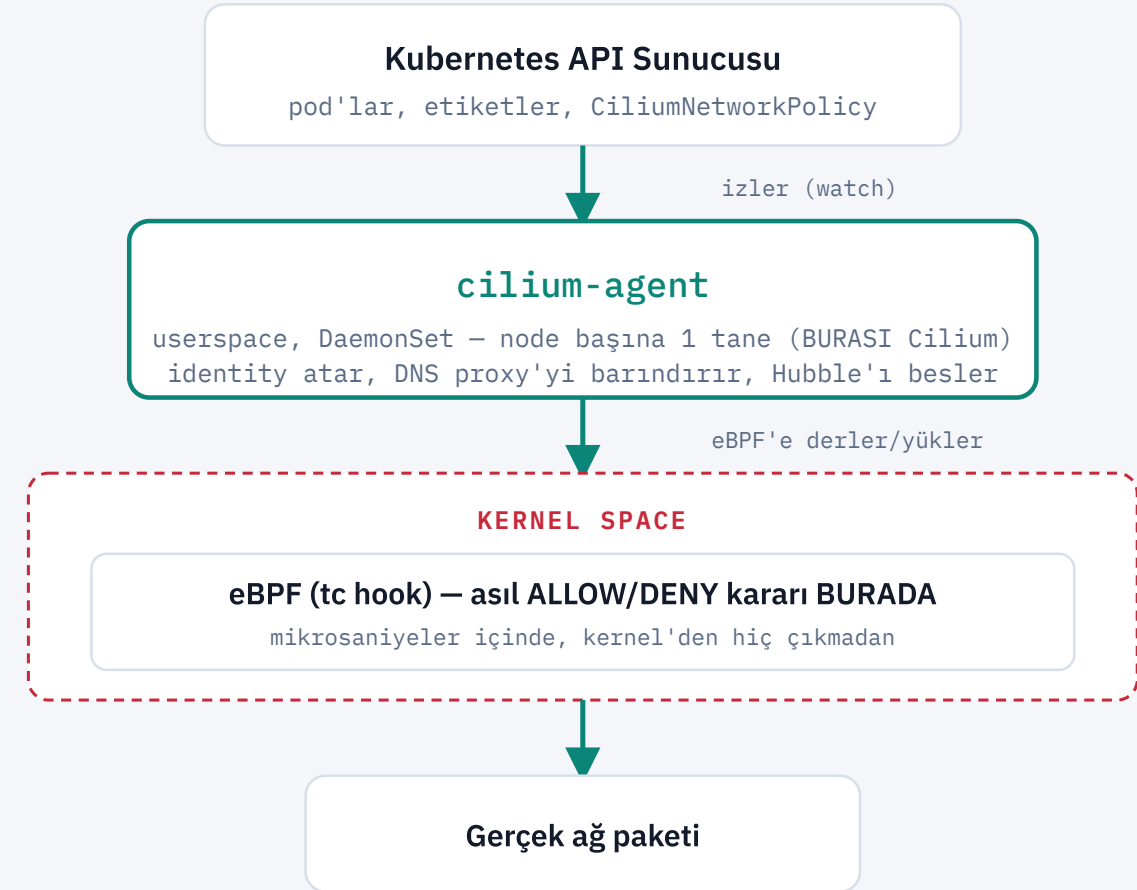


eBPF, socket ile ağ kartı arasındaki tek karar noktası — DROP kararı verilirse paket TCP/IP yığına hiç ulaşmaz.

eBPF Üzerine Kurulu Bir Ürün

Cilium, eBPF'i ALTYAPI olarak kullanan, Kubernetes ağ/güvenlik/gözlemlenebilirlik SORUNUNU çözen spesifik bir yazılım projesi. Motor eBPF, Cilium o motoru kullanan arabalardan biri (bpftrace/Falco/Katran gibi diğerleri de var).

Bu sunumdaki her kavram — **CiliumNetworkPolicy**, kimlik modeli, **toFQDNs**, **Hubble** — Cilium'a ait, eBPF'in kendisine değil.



Cilium = **cilium-agent** — Kubernetes'in bilgisini eBPF'e çeviren, node başına çalışan userspace köprü. Asıl karar bir kat altında, kernel'de.

YAML'dan Kernel'e

- 01 Biz **YAML** yazarız — CiliumNetworkPolicy
- 02 **cilium-agent** (node başına 1, Kubernetes API'sini izler)
- 03 YAML'ı **eBPF bytecode**'una derler
- 04 Pod'un ağ arayüzüne (**tc hook**) yükler
- 05 Paket gelince kernel anında çalıştırır → **ALLOW/DROP**

Pod

En küçük birim — tek IP paylaşan container grubu.

Service

Bir grup pod'un önündeki sabit isim+IP.

Node / Cluster

Tek makine / bunların bütünü.

Workload

Pod'ları nasıl çalıştıracağını tanımlar (Deployment/Job).

Hubble, aynı eBPF altyapısını GÖZLEMLEMEK için kullanır — her kararı "flow" olarak akıtır (**verdict: FORWARDED/DROPPED/...**; **identity/ip/fqdn/port/http-*** ile filtrelenebilir).

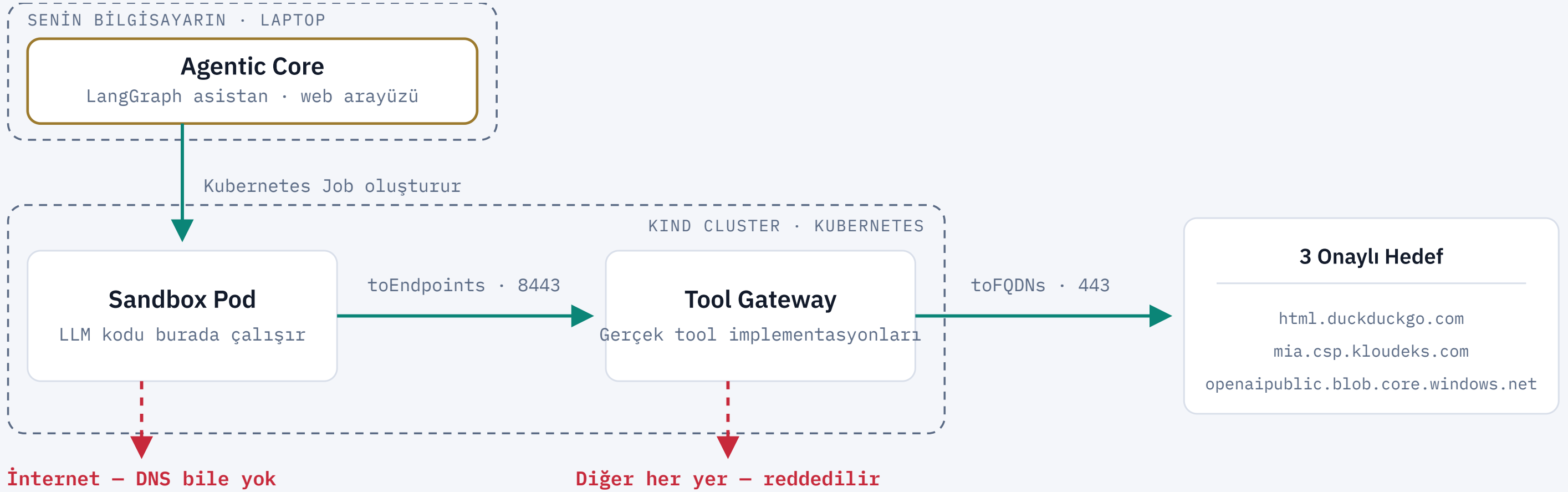
Kod Yazabilen Bir Asistan, Sızdırabilen Bir Asistandır

Güçlü bir AI agent yalnızca metin üretmez — kod yazabilir, dış sistemlere bağlanabilir. Sınırsız internet erişimi, üretilen her kod satırını potansiyel bir veri sızıntı noktası hâline getirir.

"Sandbox/agent ortamlarının dış ağ erişimi eBPF/Cilium ile merkezi olarak kontrol edilir. Sadece onaylı tool/API kanallarına erişim verilerek veri sızıntısı engellenir."

— Principle II, PTC Egress Policy

İki Katmanlı Savunma



Katman 1 kanıtı

Sandbox DNS'i bile çözemez — `evil.com`'a bağlantı denemesi CoreDNS'e ulaşmadan (16 ayrı DROPPED paket) düşer.

Katman 2 kanıtı

Tool Gateway'de DNS serbest ama TCP sadece 3 FQDN'e — `example.com` denemesi "Network is unreachable" ile biter.

Karar tam olarak ne zaman

İlk pakette (SYN) — el sıkışma hiç başlamadan. TCP'nin "kurulu" durumu hiç oluşmaz.

Portlar, İzlenebilirlik, Ömür

8443 / TCP

Sandbox → Tool Gateway. Tek izin verilen port.

443 / TCP

Tool Gateway → 3 onaylı FQDN.

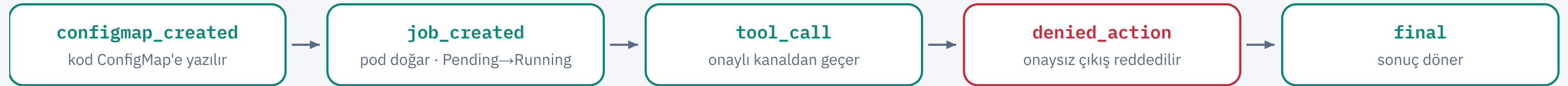
53 / UDP

Tool Gateway → CoreDNS, serbest ama TCP'den ayrı kısıtlı.

80 / TCP

Asla onaylı değil — hangi port olursa olsun aynı şekilde engellenir.

BİR SANDBOX'IN ÖMRÜ — PTC panelinde her adım canlı akar, hiçbiri gizlenmez



en fazla 30 saniye

Biter bitmez ConfigMap + Job açıkça silinir — hiçbir zaman kalıcı iz bırakmaz.

Aynı IP, Farklı İsim

toFQDNs, onaylı ismi DNS yanıtından öğrendiği IP üzerinden takip eder — ama eBPF'in paket üzerinde görebildiği tek şey hedef IP'dir, hangi ismin istendiği değil. Onaylı isimle AYNI IP'yi paylaşan başka bir isim de aynı geçişi alır.

Varsayım değil — canlı DNS sorgusuyla doğrulandı.
Paylaşılan CDN/altyapı senaryolarında gerçek bir sızıntı yolu.

```
# Tool Gateway'den iki farklı isim, canlı DNS sorgusu
socket.gethostbyname("mia.csp.kloudeks.com")
→ 185.199.89.67    (onaylı FQDN)

socket.gethostbyname("console-mia.csp.kloudeks.com")
→ 185.199.89.67    (listede YOK — ama aynı IP)

# TCP bağlantı denemesi — ikisi de
# AYNI IP'ye gittiği için...
ALLOWED — Cilium ismi değil, sadece
185.199.89.67'yi kontrol edebiliyor
```


SNI: Zarfın Üzerindeki Adres

```
toPorts:
- ports:
  - port: "443"
    protocol: TCP
  serverNames:          # yeni
  - mia.csp.kloudeks.com
  - openaipublic.blob.core.windows.net
  - html.duckduckgo.com

# TLS ClientHello'nun SNI alanı düz metin
# – Envoy şifre çözmeden okuyabiliyor
```

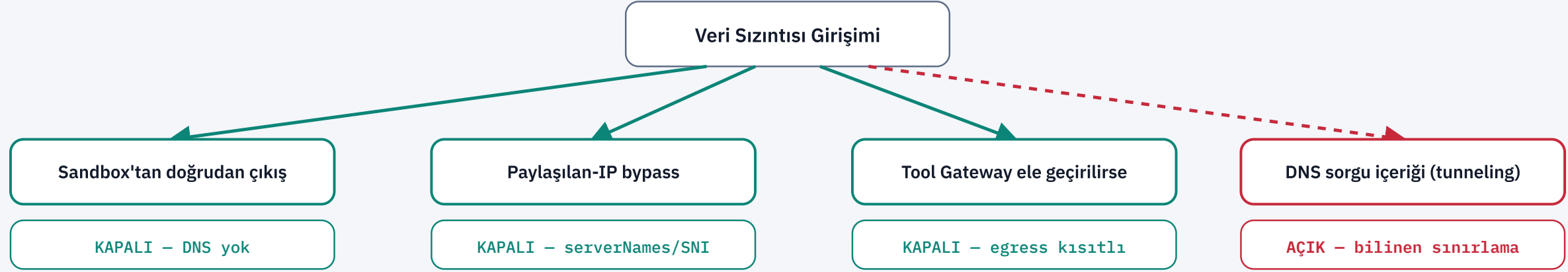
• ÖNCE: Hubble **FORWARDED** – bağlantı tamamlandı

• SONRA: SYN/ACK yine **ALLOWED** (aynı IP)...

• ...ama ClientHello sonrası **17-request DROPPED** – bağlantı TAMAMLANMADI

Paylaşılan IP hâlâ orada — L3/L4 hâlâ aldaniyor. Fark, artık bunu yakalayan bir L7/SNI katmanının (cilium-envoy) var olması. Canlı test edildi.

Dört Yol, Dört Sonuç



100 satır YAML, 0 bağımlılık

SNI düzeltmesi tek bir policy alanı — cilium-envoy zaten kuruluydu.

2 zafiyet bulundu

Paylaşılan-IP (kapatıldı) + Tool Gateway ingress açığı (düzeltme hazır).

~7sn/çalıştırma maliyeti

K8s+Cilium setup bedeli — SandboxWarmPool/Firecracker/WASM ile ~ms'ye indirilebilir (ayrı araştırma).

Şimdi Canlı Gösterelim

- 01 Sıradan + çok adımlı bir soru — PTC üzerinden, tek/çoklu tool
- 02 Kaçış denemesi (DNS seviyesi) + `fetch_url` ile Tool Gateway'in kendi bloğu
- 03 Prompt-injection senaryosu — agent kansa bile engellenir

Sandbox, hiçbir zaman "kısıtlı internet" görmüyor — tek bir onaylı kapıdan başka hiçbir şey görmüyor. Geri kalan her şey, o kapının arkasında, kendi sınırlarıyla duruyor.

**Erişim değil,
kanal onaylanır.**

Aynı Güvenlik, Daha Az Maliyet?

BİR PTC ÇALIŞTIRMASI — 7.21 sn (canlı ölçüldü) · ilk 0.03 sn K8s API yazmaları



3.08 sn · Scheduler + Cilium CNI

(IPAM + identity + eBPF attach)

↑ Alternatiflerin hedefi tam olarak bu kısım

4.10 sn · Poll aralığı + cleanup

kendi implementasyon detayımız — ayrıca düşürülebilir

SandboxWarmPool

~%90 ↓ · **saniye altı**. Ağ hazır pod havuzundan "claim" edilir. Kubernetes SIG-Apps'in resmi projesi — hazır CRD.
Bedel: boşta duran pod'lar kaynak tüketir.

1 Pod / N Container

CNI bedeli N'e bölünür. Aynı IP, aynı identity → politika hepsine otomatik uygulanır.
Bedel: "boş slot" takibini kendin yazarsın.

Firecracker microVM

p50 3.2 ms restore. Hazır microVM'in bellek anlık görüntüsü geri yüklenir (AWS Lambda SnapStart).
Bedel: K8s'i bırakıp kendi orkestrasyonun. Cilium K8s'siz de çalışır.

WASM / WASI

En hafif — kernel devreye girmez. Ham socket dil seviyesinde yok; Cilium'a ihtiyaç azalır.
Bedel: Python'u WASM'a taşımak başlı başına problem.

DONANIM AYAK İZİ — ÖLÇEKLENEBİLİRLİK AÇISINDAN

500m CPU · 256Mi RAM

Bir çalıştırmanın **sert tavanı**. requests tanımlı olmadığı için Kubernetes bunu limits'e eşitliyor — yani scheduler bu kaynağı peşinen ayırıyor (canlı doğrulandı).

~30 eşzamanlı sandbox

Bu tek node'da (16 vCPU · 15 GiB) **CPU-bağımlı tavan**. RAM ~60'a, pod yuvası 96'ya yeterdi — darboğaz CPU. Ölçek = node eklemek.

Süresiz tüketim mümkün değil

activeDeadlineSeconds: 30 · backoffLimit: 0 · ttlSecondsAfterFinished: 300 · tur başına en fazla **2** çalıştırma. Kötü niyetli kod en fazla 0.5 çekirdeği 30 saniye meşgul edebilir.

CANLI ÖLÇÜM — "1 Pod / N Container"ın gizli maliyeti

Aynı pod'daki container'lar birbirine localhost üzerinden ulaşır. Bu trafik veth'ten geçmediği için eBPF hook'una **hiç uğramaz** — Hubble'da arattık: **0 flow**. Cilium bu kanalı ne görebilir ne engelleyebilir. → Bu model seçilirse slot'lar **hiç port açmamalı** (pull modeli). Bugünkü "her run = ayrı pod" tasarımında bu kanal yapısal olarak yoktur.